

Migrations Done Well

A guide for executing migrations well, at both small and large scales.



GERGELY OROSZ

MAR 15, 2022 • PAID



73



Share



Migrations are one of the most overlooked topics in software engineering, especially at high-growth startups and companies. As a company's operations grow, new systems and approaches are adopted to cope with extra load, more use cases, or more constraints. From time to time, engineers need to migrate over from an old system or approach, to a new one.

And this is where things can get interesting, unexpected... and even ugly.

This issue is an extensive guide on the process of migration. You can use it as a blueprint when preparing and executing migrations. This guide covers:

Part 1: Typical migrations

1. [The stories of four different migrations](#)
2. [Types of migrations](#)

Part 2: Executing a migration

3. [Preparation](#)
4. [Pre-migration](#)
5. [The migration](#)
6. [After the migration](#)
7. [The migration's long-tail](#)
8. [A migration checklist](#)

[Part 3: The people and the business side of migrations](#)

9. [The people aspect of migrations](#)
10. [Selling migrations to the business](#)
11. [Closing advice for migrations](#)
12. Further reading

If you're not yet a full subscriber, you can read [Part 1 of the series here](#).

Part 1: Typical migrations

1. The story of four different migrations

Each migration is a story in itself, so here are four migrations. Each is distinctly different from the others.

The migration with planned downtime

In 2012, PayPal [announced](#) it would take all of its global payments processing systems down for an hour, within a three-hour window. This was to be while it moved over to a new infrastructure. It was decided that taking the system down, moving data over to the new infrastructure and then spinning it up, was the safest way to go.

Even though the scheduled time was at night in the US, this migration caused losses thanks to transactions by global customers not being processed during this time. However, the migration did complete as planned, while the disruption and losses to the business were within the realm of what PayPal had planned for.

The migration gone terribly wrong

UK bank TSB has 5 million customers. It performed an IT migration in 2018, which ended up locking close to 2 million customers out of their accounts, unable to withdraw cash. Fixing the issues of the migration took many months; five months after the

migration went wrong, some customers were still locked out of their account, and the CEO of the bank was forced to [quit](#) over the episode, after seven years in post.

Although the root causes are still opaque, it seems this disaster was down to a lack of proper testing ahead of the migration. [Press covering the incident](#) mentioned configuration problems, lack of infrastructure capacity and coding issues, all contributing to the problem. The outage cost the bank at least \$480M in costs, not to mention heavy reputational and business damage.

The zero downtime migration

When I joined Uber in 2016, the company had two payment systems: one processing pay-ins from customers (Riders and Eaters) and one dealing with payouts for partners (Drivers and Couriers). We started building a new, single payment system to replace these two standalone systems. In 2017, we switched over to the new system – codenamed Gulfstream – to handle Uber’s payments.

The switch to a new payments system was invisible from the outside and the migration was done without downtime or major issues. This was a case of a zero downtime migration executed well.

However, the long tail of the migration stretched on for years, and this long tail later turned out to cause problems.

The migration long-tail causing an outage

Although the new payments system worked as the primary system, Uber still had dozens of internal systems that were dependent on the old payments infrastructure. The new system did writebacks to these old systems to support consumers that had not yet migrated over.

Long migrations can be risky and this case was no different. In 2018, drivers were [unable to use Instant Pay for several days](#). The reason was an outage in the writeback system which pushed production data to the legacy system. The system which handled driver payouts still used this old system, and as writebacks were not arriving, this caused problems with payouts.

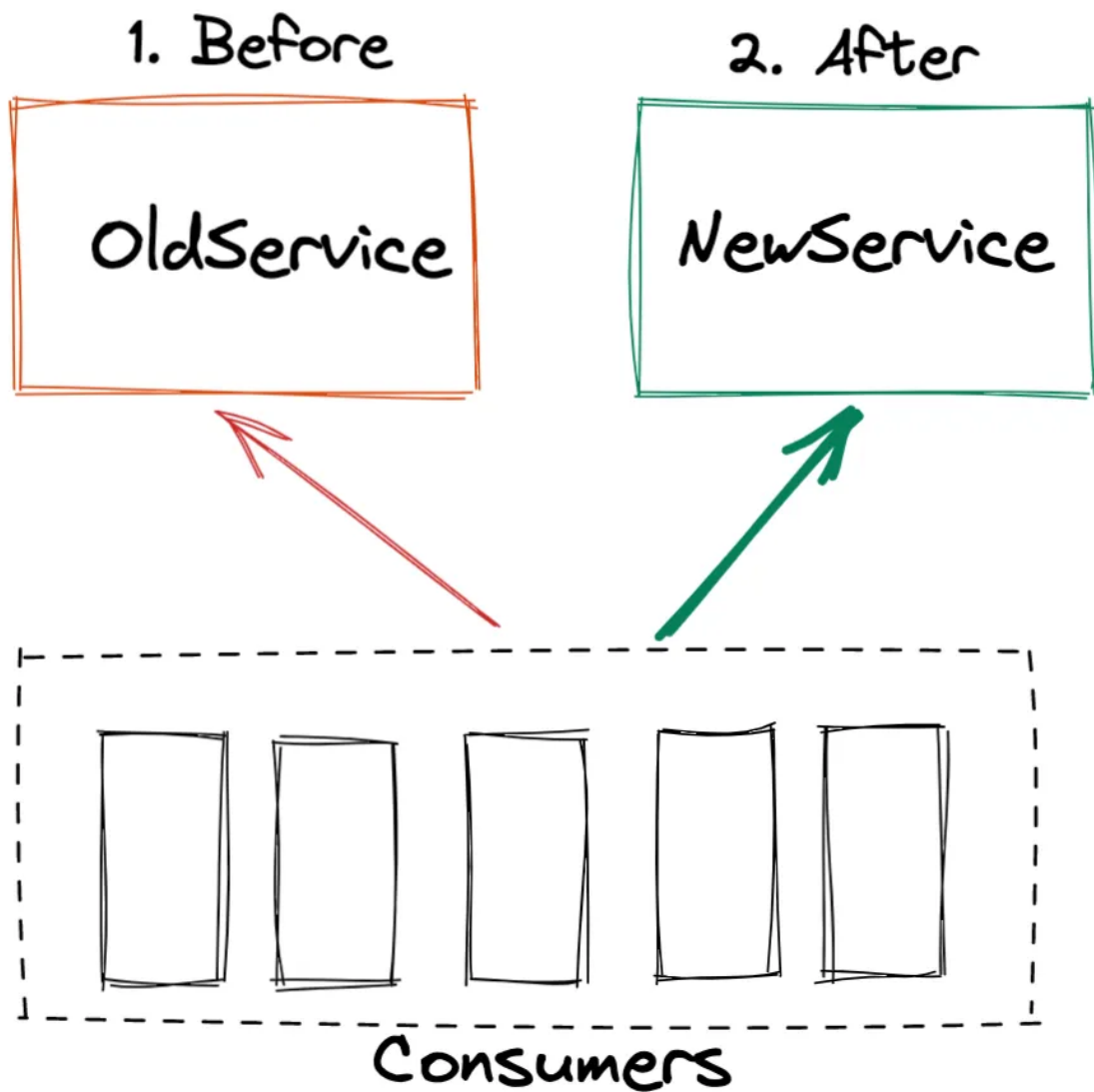
The engineering team didn't immediately detect the outage because while the production payments system had robust monitoring, the writeback system – meant to be in place for months and not years – did not have sophisticated monitoring and alerting.

Migration long-tails are common, risky and often cause outages, just like they did in this case.

2. Types of migrations

The word 'migration' describes a broad range of activities. Here are the most common types of migrations we engineers come across.

Service replacement. Replace OldService with NewService and move all customers over to eventually use NewService. An example is replacing an old, legacy payments system with a new one like Uber did, in the example above.

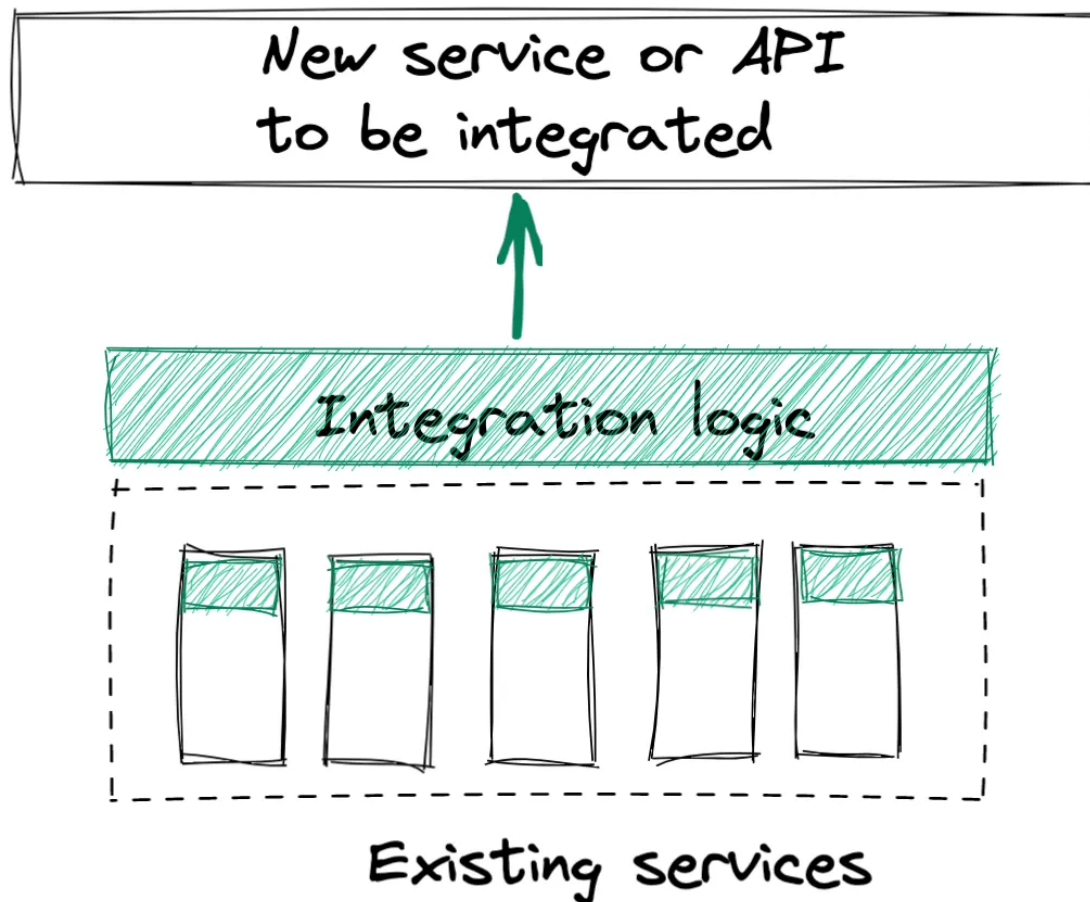


newsletter.pragmaticengineer.com

This type of migration is very common at any fast-growing company. There might be many reasons to write a new service instead of improving the old service, such as:

- The old system not supporting enough business requirements.
- The need to improve nonfunctional characteristics like handling extra load, achieving lower latency, storing more data.
- The language or framework the old service was written in, is no longer supported or preferred within the organization.

Service integration. Integrate a new service or approach.



newsletter.pragmaticengineer.com

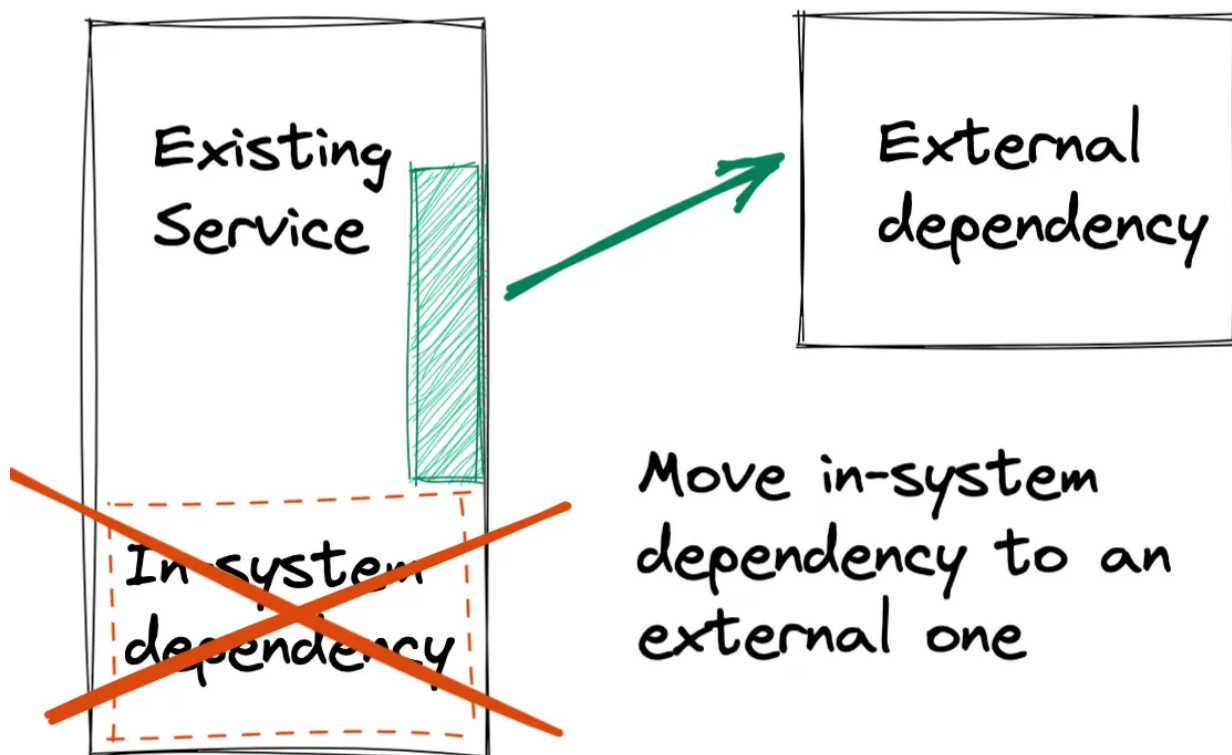
Examples include:

- Integrate a new API for use. For example, integrate a new payments processing service.
- Migrate to a new version of an external service. For example, move from the v1 API of a provider to a v2 API.
- Migrate to a new version of a service where instead of polling for events, the new service pushes events. Or instead of synchronously consuming inbound events, inbound events are consumed asynchronously, triggering callbacks.

As the examples above show, service integration migrations can include many things. There are engineers who might not even consider some of the above to be migrations, but treat them as a change in functionality. But I argue that we should treat them as migrations.

This is because these are changes where an external service alters, and your code also changes to ensure correct future behavior. This characteristic is what makes them risky changes, and why it's wise to proceed with caution, following a migration playbook.

Service extraction. Move from an in-system dependency to an external one. This is a common approach at high-growth startups when large services get split up into smaller ones.



newsletter.pragmaticengineer.com

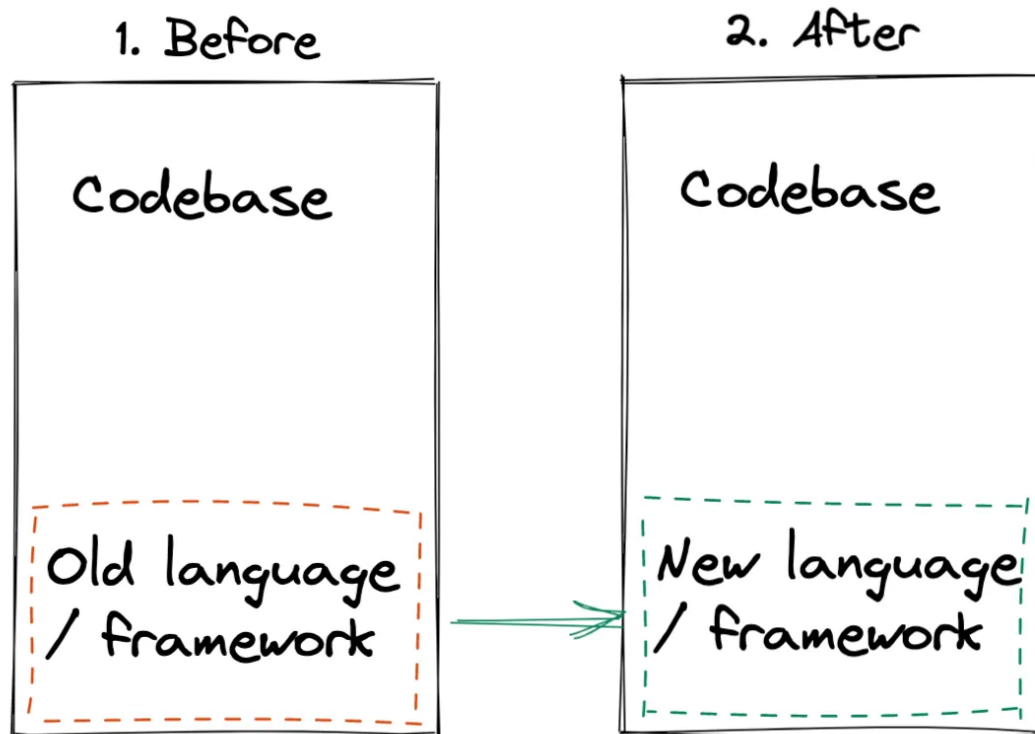
On the surface, service extractions seem like some of the most simple migrations. After all, we're just moving code that already works into a new service, right?

Wrong.

Service extractions are much more risky than simply moving code around. There are risks around networking and the correctness of the new service; especially as the code is rarely copied and pasted. Also, extracting a service comes loaded with future operational risks.

While service extractions are one of the easiest types of migrations to pull off successfully without any monitoring or alerting in place, I suggest to avoid this temptation and to follow a more thorough migration runbook, like that we discuss below.

Code migrations. Moving a part of the codebase to use a different framework, library or a programming language.



newsletter.pragmaticengineer.com

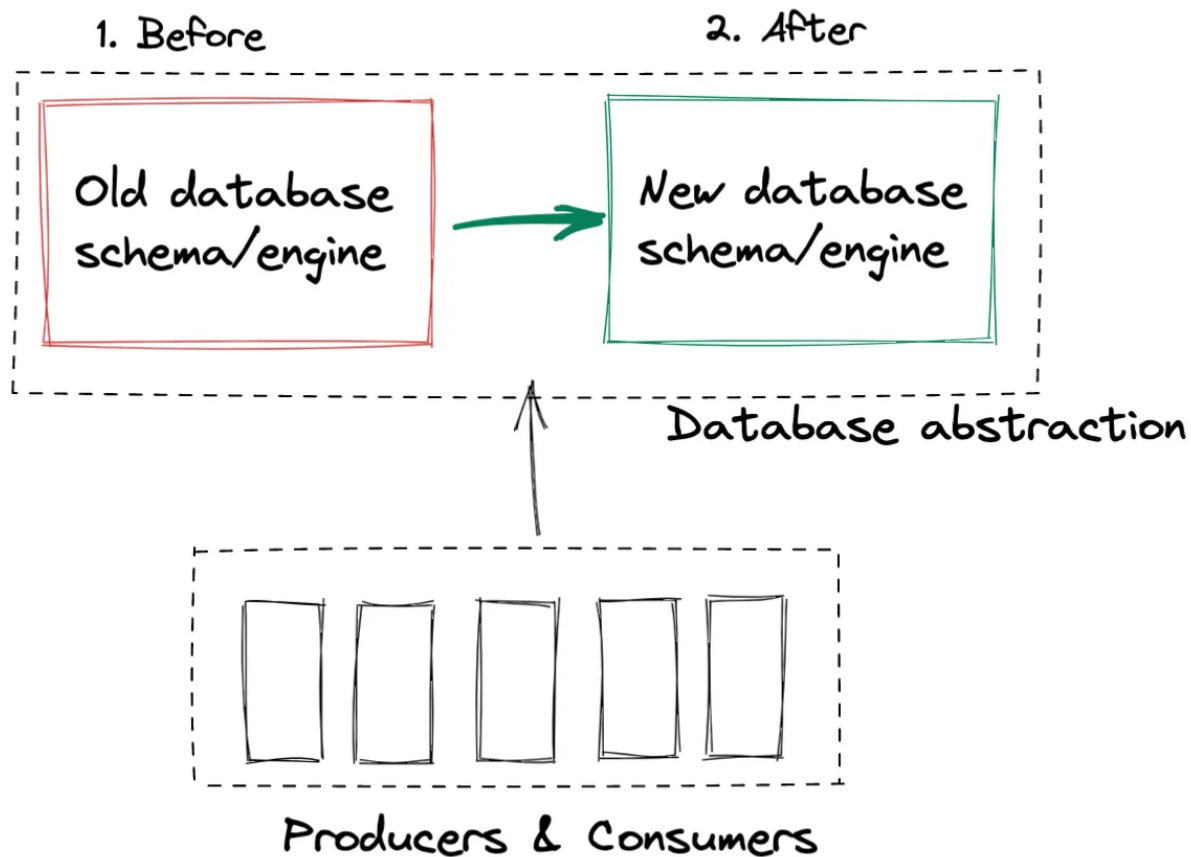
The biggest challenges of this type of migration are:

- **Large-scale change.** It's not uncommon to have to change a very high number of files and lines of code.
- **Hard to rollback.** Because the change is so large, it's also difficult, or sometimes impossible, to easily roll back the change. This is the opposite of most code changes which are small and can be unwound very easily.
- **Harder to test in production.** As the change is large-scale, it can be harder to test the full change under production traffic.

A few examples of code migrations:

- Migrating to a new language. For example, Stripe [migrated](#) 3.5M lines of code over a weekend, from JavaScript type checked by Flow to Typescript.
- Upgrading to a new major version of a library, which introduces API-breaking changes.
- Changing coding patterns across the codebase. For example, updating a JavaScript codebase to use [promises](#) instead of callbacks.

Data migrations. Migrating one or more databases to a new schema or database engine.

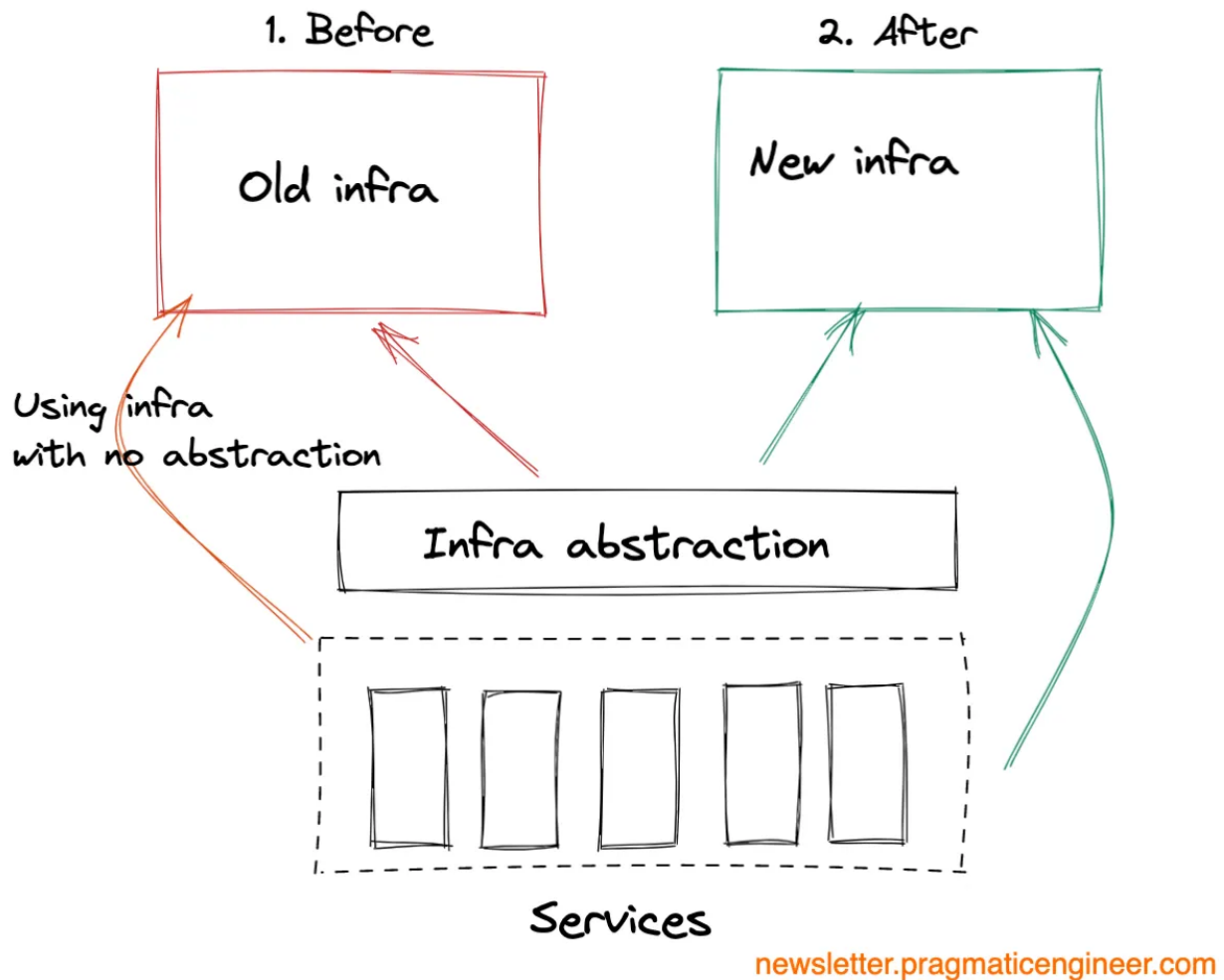


newsletter.pragmaticengineer.com

Data migrations are one of the most risky types of migrations. There are several reasons why::

- **Difficult to roll back.** When code is changed, it's easy enough to roll back a migration by rolling back the code. But this isn't always the case with data.
- **Difficult to do in an atomic way.** Ideally, a data migration is done as an atomic step. From one moment to the next, the old database or schema stops being used and all data moves to the new database or schema. But in practice, moving data takes time. This is why it's much easier to do a data migration with downtime.
- **Data migration is often tied to code changes.** For the migration to work as expected, code changes often need to be made at the very same time as the data is migrated.
- **More edge cases to worry about.** You will need to worry about edge cases like producers writing to the old database or using the old schema, or consumers not being able to understand the new schema.

Infrastructure migrations. Move storage, computing or networking infrastructure to a new provider or a new location.



Infrastructure migrations are the most risky, as they impact all services using that infrastructure. An infrastructure migration gone wrong can take down all dependent services with it. For this reason, these are the ones that typically need more preparation than even data migrations.

While all other migrations are virtual in the sense that they typically involve moving code or data, infrastructure migrations are when the physical infrastructure might also need to be moved or changed, and onsite work might be needed. Although most companies use cloud computing providers like AWS, Azure or Google Cloud, plenty of companies still operate their own data centers, or run their own server boxes. Infrastructure migrations can mean changes to these as well.

Service and tool upgrades are also changes which can cause outages. Examples of this include upgrading the CI/CD system to the latest version, updating JIRA to a new version, and other tooling infrastructure changes.

But for upgrades, the vendor of the tool is the one performing the migration. It's advisable to understand how this migration happens under the hood and if it's a high-risk upgrade, and to use the approaches in this article to reduce the risks of the update.

The focus of this article is service migrations, and we'll have a section on data migrations as well. Infrastructure migrations are out of scope, both because of the complexity, and also because they're also the least common type of migration.

Part 2: Executing a migration

3. Preparing for migrations

Migrations are risky and when they go wrong, they can cause all kinds of significant damage. However, if you do some groundwork before starting the migration, you'll reduce risk, gain confidence and understand the scope of the migration better. Here are a few things you should do:

- **Understand the reason for the migration.** Why is this migration needed? What is wrong with the old solution? What is the impact of the old solution being unsuitable? What would happen if this migration did not complete?
- **What were the constraints in the past?** When a migration is needed, it's because the current solution fails to meet a particular requirement. This could be availability, performance, or something else. Outline what these capability constraints are. You'll need to make sure the new system solves for these past constraints.
- **A map of the differences of Old vs the New.** Outline the old system vs the new system. What is changing and why, and how do these systems work? Are external interfaces changing at all? Are you adding or removing functionality for the new system?
- **Consumers and producers needing migration.** Which customers of the old system need to be migrated to the new one? Map customers of the old system. Will all consumers move over, or will some remain on the old system?

- **Write and share a migration plan.** Write up how the migration will happen. What are the migration steps? Who will do each step? For example, who will build the new service, who will migrate each of the consumers, and who will monitor the progress of the migration? Put this plan in writing and share it with consumers whom the migration will impact.
- **Capacity planning.** What load is the current system under? What load will the new service need to handle? For load planning, don't only look to the time of the migration, but look ahead into the future. How will you ensure the new system has the capacity to handle this load? How will you confirm it can handle the load?
- **Error budgets.** For complex and large migrations you can expect some things to go wrong. You might also have migrations where you expect a small percentage of traffic to not work as expected. Instead of just accepting things going wrong, budget for it. What is the acceptable limit for errors during the migration? This could be defined as latency degradation, % of data that is not consistent for a certain time, percentage of application errors during the migration, and so on. By defining error budgets you'll also have to measure them to keep yourself honest.
- **Edge cases.** What are potential tricky situations you need to take into account? Write these up, then share it with the team working on the migration and all consumers impacted, so they can also add their edge cases. Common edge cases you should worry about include:
 - What happens with data producers during migration? For example, say you have processes writing to the database. What happens when the migration happens? Will these writes continue? Will you stop them? Will they be queued?
 - How will service consumers behave during migration? Will there be edge cases when clients behave in unexpected ways, or experience data consistency issues as they interact with a system undergoing migration?
- **Security audit of migrations.** What are the attack vectors during and after the migration? Are there vulnerabilities a bad actor could take advantage of, as the migration happens? Vulnerabilities might not be strictly related to the code or data, they could include phishing attacks.

This is exactly what happened to NFT marketplace OpenSea, in February of this year. The company announced a week-long migration to customers. A bad actor realized this migration was an opportunity for an attack and sent out emails that looked like they were from OpenSea, asking customers to take action for the migration to happen. Customers then were tricked into [authorizing a smart contract](#) they believed was from OpenSea, and dozens of unlucky customers lost over \$1.7M in NFT assets.

Had the company predicted such phishing and sought to counter it, they could have chosen to either shorten the migration window, communicated the phishing risks, or have monitored suspicious smart contracts authorized during the migration period, and warned users against executing them.

- **Run a migration pre-mortem.** As engineering director Nathan Gould suggests:



Nathan Gould
@0xNathanGould

@GergelyOrosz Love this. One of my favorite migration practices is to hold a "pre-mortem".

The basic play is:

- Write up a migration plan
- Gather the team running the migration, plus stakeholders
- Brainstorm stuff that could go wrong
- Identify and prioritize ways to mitigate risk

5:32 PM · Mar 11, 2022

3.1 Preparing for data migrations

Data migrations bring additional complexities of their own and tend to require more thorough planning. Some additional complexities you might need to consider are:

- **Too much data to migrate in one go.** In the case of service replacements, this means that the new service might need to still use the old service to look up older data. In this scenario, smart approaches could be used. For example, when

modifying a record, the new service might create a copy from the old system, and stop using the old system for that record.

- **How will data producers be migrated?** What happens with data-producing processes, for example, processes writing to the database at the time of the migration? Will they be paused? The writes queued? The writes dropped?
- **Active-active challenges.** When using an [active-active data storage setup](#) – distributing data across several clusters for resilience – you'll have additional challenges when migrating. If migrating with downtime, you might be able to migrate all clusters during this downtime. However, if running a zero-downtime migration, consumers will see inconsistent data. How will they deal with this inconsistency?
- **Rollback.** If the migration has issues, how will you revert to the previous state? When utilizing downtime to perform the migration, the rollback might be as simple as not switching over to the newly migrated data. However, with zero downtime migration, you might need tooling to write back data committed to the migrated database as you roll back to the old database.
- **Disaster recovery considerations.** In the event of a major issue like data corruption or a ransomware attack, how easy is it to roll back to a pre-migration state of the system? Are there sufficient backups? Should you make a last snapshot save before the migration proceeds?

4. Pre-migration steps

You have a plan in place and are confident you have the edge cases covered. Can you start the migration? Almost certainly not yet! First, you'll need to ensure you have monitoring in place so you can track the status of the migration and detect problems. You'll also want to validate that the migration will work with shadowing, dry runs and other processes.

4.1. Monitoring

Monitoring the migration is the single most important action which can make a migration successful, and detect one going wrong. The lack of dedicated migration

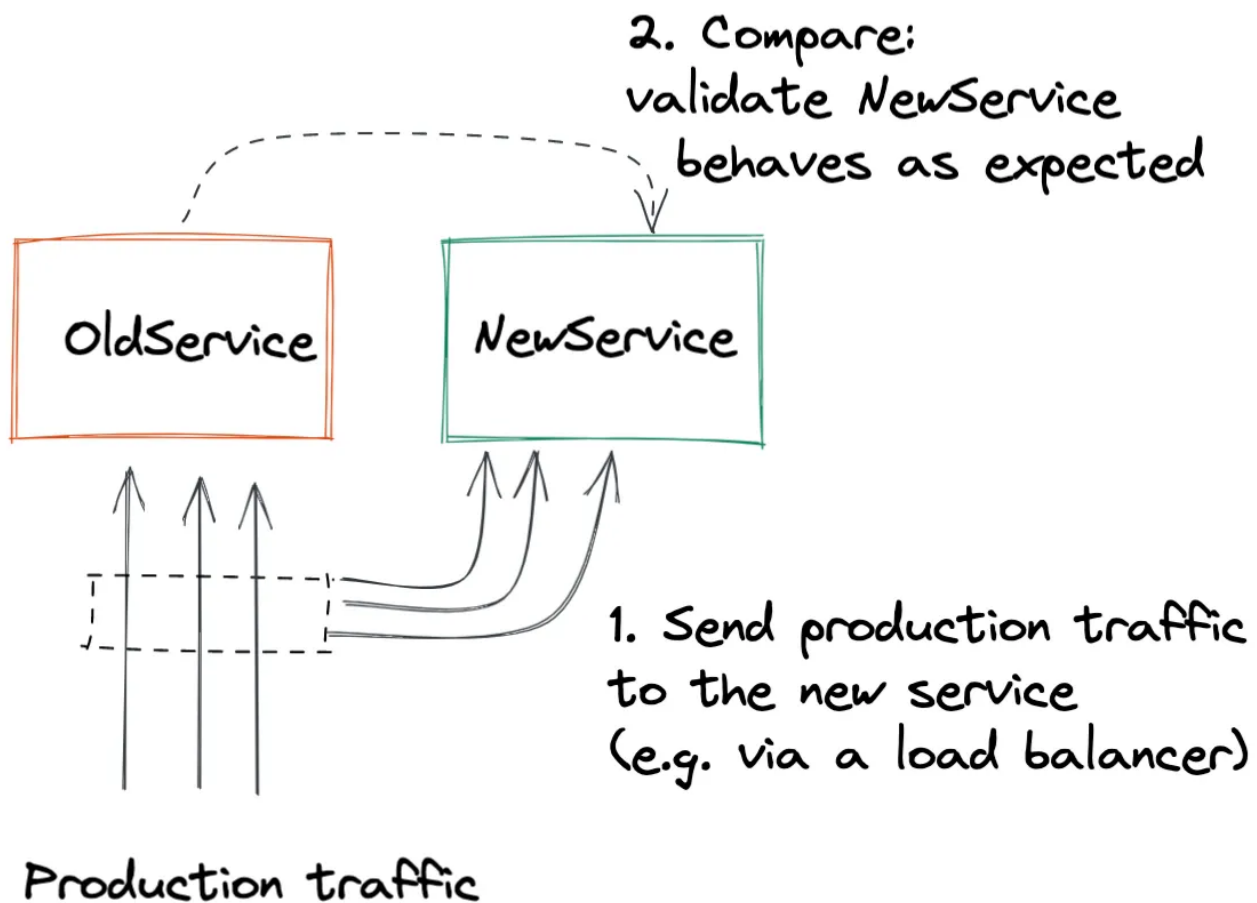
monitoring is the reason for most migrations causing outages, in my experience.

- **Define what needs to be monitored.** How can you tell if the migration goes well? What do you need to measure to know that customers are not being impacted and the service is healthy? For data migrations, how can you measure that the data is not corrupted?
- **Build the monitoring systems.** Put in place the graphs, tools and alerts that tell you if the migration is healthy or if it has issues.
- **Throwaway monitoring is to be expected!** Many engineering teams are hesitant to build monitoring as they're used to building graphs and alerts for permanent features. Break away from this thinking! You need to build monitoring and yes, it will be removed once the migration is done.
- **Do you have monitoring for all error budgets?** Remember how you defined your error budgets? Make sure there's monitoring for all of those metrics.
- **If it's painful to build one-off monitoring, take note for later.** At companies which have invested heavily in platform teams, building monitoring for migrations should be a breeze. If it's not, then you should incentivize making it easier to do. Do this by contributing to tooling, or by pushing management to invest in tools, be it via internal platform teams or contracting with vendors.

4.2. Validation

Validate that the migration will work as expected. Once you've completed most of the development work for the migration, you'll want to hold off rolling out in production. Instead, you should validate the migration with production data and traffic. Here are some common ways to do this.

Shadowing, also commonly referred to as 'parallel run' or 'shadow loading.' This is a common approach, both with service replacements and service integrations. This approach means sending all production traffic to the new system or new integration as well, and monitoring that it works as expected.



newsletter.pragmaticengineer.com

Shadowing production traffic.

Shadowing has several benefits:

- Migration issues are caught early.
- It's as close to pre-production testing as it gets.
- It serves as load testing. If your shadowed system can handle production load, you can be confident that it won't have issues when switching over.

There are a few caveats with shadowing:

- Shadowing validation. You'll likely need to build shadowing validation tooling to confirm the new system works as expected.

- Mocking might be needed to avoid side effects. The new system might need to mock certain functionality. For example, if you are migrating an old system which also sends emails in certain scenarios, you'll want to stop the new system from sending emails while in shadowing mode. Doing so would result in emails sent twice!
- Not always practical. There may be times when shadowing is not an approach that's pragmatic. This is the case when you would have to mock much of the shadowed system's capability. For example, if you are migrating the email layer in an application, shadowing email sending without sending emails might be meaningless.

Load testing is a common approach to confirm that the new system has the capacity to handle future, increased loads. While shadowing can give a sense of current load-handling capability, the goal of load testing is to simulate extreme loads.

Common ways to do load testing are these:

- **Use mocked or generated data.** Generate test data and execute a load test. The benefit of this approach is that you can tweak the load test characteristics easily. The downside is that the test data will not exercise edge cases which only real-world data has.
- **Use production data, but in bulk.** A more common load testing approach is to sidestep test data and use real data. Collect production data for a while, then use this production data for load testing.
- **Combine shadowing with a bulk release of production data.** We used this approach at Uber with a system called Hailstorm. It buffered all production requests coming in for a defined time and replayed it in a shorter time window. For example, we released 10 hours worth of production data in one hour.

Performance sampling is an approach worth doing for high-load systems, especially when you are changing technologies like frameworks or programming languages. It's easiest to do when combining with shadowing. What are the performance characteristics of the new system, compared to the old? How has latency changed, and has resource usage like CPU utilization and memory usage, decreased or increased?

Do a dry-run of the full migration. Why wait to do the migration in production, when things may go wrong that cause outages? If possible, do a dry-run migration and inspect whether things work as expected.

You can combine a dry-run migration with shadowing. Dry-runs are common to do with data migrations, where this might be the best way to confirm that data moves as expected.

Test for events that will only happen in future. As an edge case, your migrated system might have to deal with edge cases that only happen on certain future dates. For example, if migrating a billing system that sends out bills at the end of the month, you'll want to test that this functionality works.

One option is to do shadowing for long enough so that these future events occur, and you can validate them in real time. However, you might not have the luxury of waiting, especially if these events happen once every few months, or once a year. In this case, you'll have to simulate these events and validate that the migrated system handles them as expected. A good example is testing billing systems like this:



Jeroen Baidenmann
@jeroenbai

@GergelyOrosz Pre-migration:

- do a complete data migration to the new database and test new service features + performance with real life (anonymized) data.
- test future processes. E.g. billing run every 1st of the month: do it in the new system simulating future date and compare results.

9:28 AM · Feb 18, 2022

5. The Migration

We've planned the migration, done pre-migration validation and perhaps even shadowed the new system, assuming it's practical to do. Time to start the migration!

Migration downtime

The most important decision in any migration should be whether or not to have downtime for consumers of the system being migrated. This decision will determine strategies you can use for the migration itself.

Zero-downtime migrations are when customers notice nothing of the migration. These are the ones that need the most preparation to achieve.

Doing zero downtime migrations is often a steep learning curve and involves more upfront work, when done the first time. However, at companies where zero downtime is the norm, the amount of additional work drops over time as people get better at it, and at building or utilizing tools which aid the process.

There will be many cases when zero downtime is too expensive to do, in terms of time spent building tooling. There will also be cases where it's not possible, for example, with infrastructure migrations. However, if you start by considering what it takes to do zero downtime migrations, then you can make a more informed choice.



Bryan Liles
@bryanl

@GergelyOrosz The first thing you should do is determine if you need zero downtime migrations.

4:29 PM • Mar 12, 2022

I would encourage teams to do at least one zero downtime migration, if they've never done one before. Teams that have never performed such a migration often overestimate their complexity, and underestimate the benefit of not having to work outside business hours when doing zero downtime migrations.



Felix Huttmann
@felixhuttmann

@GergelyOrosz A benefit of zero downtime is being able to do all things calmly rather than in a hurry and not needing to time anything to a maintenance window. Especially in async work cultures, or cultures where stress and agitation is disliked, that is valuable.

6:43 PM · Mar 11, 2022

Migration with downtime affecting few to no customers is when you take the system down, but customers don't notice anything happened. How is this possible?

For one, your business might be intentionally offline during certain hours. For example, if you have systems servicing the stock market which only operate during business hours, customers won't notice migrations done outside these hours.

If your migration involves non time-sensitive functionality, you can also get away without customers noticing. In these cases you might be able to take the old system offline, queue incoming requests, do the migration, then replay those requests for the new system to process. For example, if you are migrating a system that sends out marketing emails, customers won't notice – or care – that some marketing emails arrive an hour after they usually do, thanks to the migration.

Migration with reduced functionality is an approach where you don't introduce downtime to a system, but some functionality will go offline. A common case is to turn a system to 'read only' mode during a data migration; all read operations still function, but new data can not be written. That data will either be queued for later writing, or it might be discarded until the migration is complete.

Migration with planned downtime is an approach where you take the current system fully offline, perform the migration, then bring the migrated system online. You are able to accurately estimate the downtime needed and can communicate this downtime, ahead of time. You typically choose outside peak hours to perform the migration, usually in the middle of the night for most customers, or at the weekend.

Migration with downtime has the benefit that you don't need to worry about edge cases of consumers accessing a system that is not fully migrated.

The downside of this approach – of taking the system offline – is that it adds pressure; if something goes wrong with the migration, there's not much time to fix it without exceeding the communicated downtime window.

Migrations utilizing regular downtime periods is an approach common at more traditional businesses which have been doing migrations for years, or decades. Several banks fall into this category, which commonly allow for downtime to happen over the weekend or during bank holidays.

Having regular downtime periods that can be used for migrations is convenient for engineering teams. They can use these often long timeframes and not have to worry about a migration going wrong as they have ample time to revert it, and to get it right in the next period.

I personally find regular downtime periods tempt engineering teams to take the easy route of not preparing well for migrations, and disincentivize zero-downtime approaches. On top of this, regular downtime periods incentivize working outside business hours like at weekends or late at night.

Migration strategies

How will you perform the migration? Here are the most common migration strategies:

Switch over. With a flip of a switch, or more often a configuration change, you route all traffic to the new system. This is usually done after extensive shadowing.

There are migrations where a switch over is the only sensible strategy. Code migrations are one of them, and data migrations might be, too. Migrations that utilize downtime are almost always ones that use a switch over method once the migration is complete.

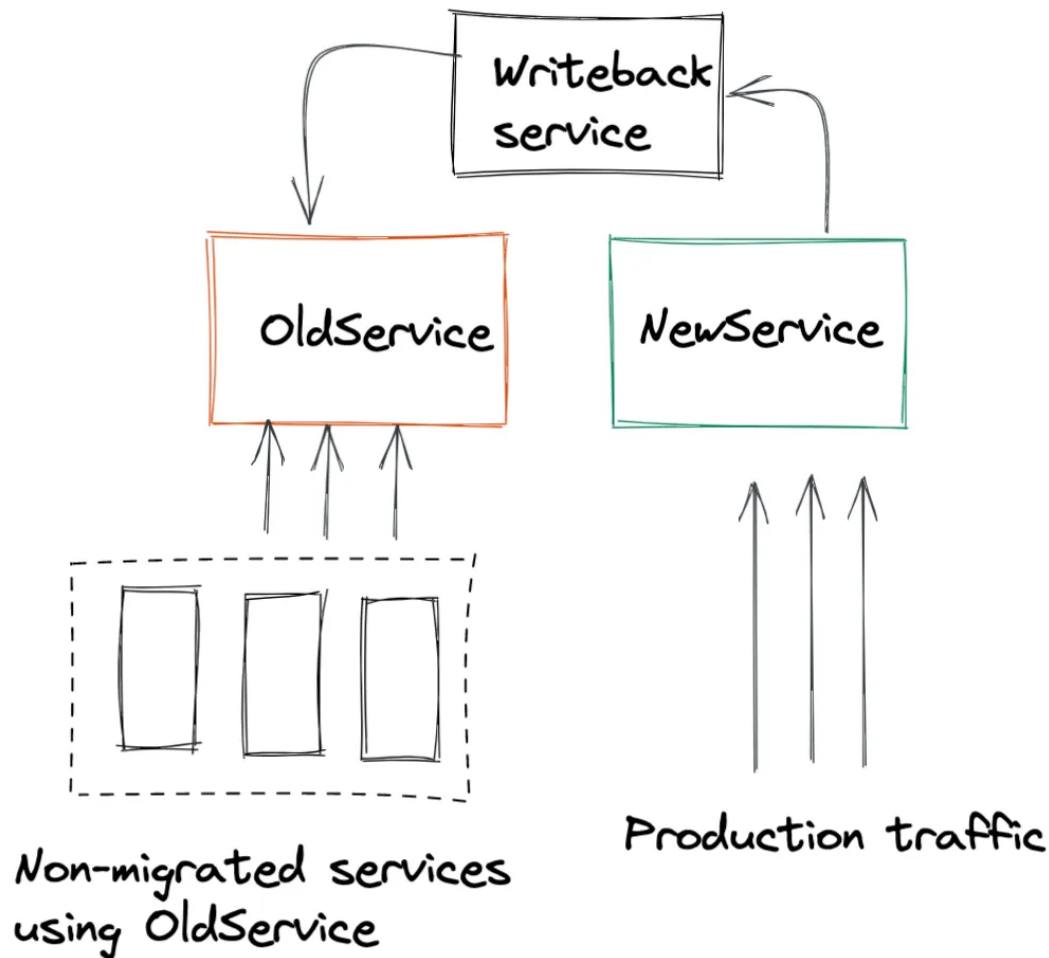
Staged rollout. This means rolling out the migration gradually to parts of the system, or to a certain group of consumers. As the rollout proceeds, the team monitors the system to make sure it works as expected, and pauses or reverses the rollout when they see issues.

Staged rollouts are popular when releasing new features, gradually rolling them out to all users. This means that many teams already have access to the tools – like feature flags – to use for staged rollouts.

Several types of migrations can easily be done as staged rollouts. However, more complex ones like data migrations or infrastructure migrations often require extra complexity to be added, in order to use a staged rollout approach. This is because in those migrations, both data migration and code changes often need to be tied together. A staged rollout with a data migration might mean writing more code to keep the migrated data and the code executed in sync; this new code is yet another source of bugs.

Writeback to the old system is a common approach with both service replacement migrations and some data migrations. In cases where the existing service has many internal consumers, the migration to the new system does not move these consumers over to the new system.

Instead, the new system writes data back to the old system, allowing for consumers to operate without changes. Now, the migration is complete in the sense that the new system operates as primary. However, there will be a long-tail migration effort to move all consumers of the old system to use the new system.



newsletter.pragmaticengineer.com

Writing back to the old service, allowing non-migrated services to keep operating without interruption.

Migration toolset

Let's get to the migration! Here are approaches to consider using.

Go through your migration plan with the team, once more. This should be a document outlining:

- Each migration step.
- How each step will be validated to confirm it's successful, so the next step can begin.
- Who does what during the migration.

- Contacts on teams who could be impacted.
- Edge cases that might occur and how those will be handled.
- A rollback plan on how to revert the migration if any of the steps go wrong.

Validate that your migration monitoring is in-place and working. The location of graphs, alerts and tools are already part of your migration plan: double check they are up. Test that they work.

Announce the timing of the migration ahead of time. How much time is enough time for this depends on the scale and impact of the migration. This could be days' worth of heads up, or weeks'. For small migrations, announcing a few minutes beforehand to relevant teams and oncalls that the migration is starting, might also be sufficient.

I strongly suggest not to start a migration without announcing it to teams and stakeholders which could be impacted. Don't forget to not only notify engineering teams, but non-engineering groups like customer support, operations and other people who can help in letting you know if they see unexpected issues during the migration.

Start! Once the announcement is out, start executing the migration.

If there's an issue: don't panic. There's always a chance that something goes wrong during the migration. If this happens, keep your cool and respond the same way as you would during any outage.

Focus on mitigating the issue, instead of fixing the root cause. Use the rollback plan in your mitigation plan. Once you've rolled back, regroup to identify the issue and prepare for a new migration.

If you don't have a rollback plan, or the rollback is not working, consider asking for a second opinion before proceeding with anything bold. You're in the heat of the moment, under pressure and it's much easier to miss things in this state. There are few situations where a second pair of eyes isn't helpful.

Writing back to the old service, so that non-migrated services remain operational.

6. After the migration

You've completed the migration. Congratulations! What now?

Announce the migration is complete. Notify not just the impacted stakeholders, but a broader group of potentially interested teams that the migration is complete.

If the migration was a non-trivial project, follow the same steps as when wrapping up projects, that I suggest in the article [Software Engineers Leading Projects](#), including the final project update:

"Write a concise summary of why people should care, what the impact – or expected impact – of the project is, summarize the work, highlight key contributors, and link to details.

"Shine a light on all team members, and anyone who meaningfully helped with the work. A final project update email is the time to praise people by name who worked on the project. Make sure to not omit key people, which is why getting feedback on this draft is a good idea."

When announcing the completed migration, specify where you'd like people to report any issues they might see as a result of the migration.

Validate that the migration is successful. Confirm that everything works as expected. Don't only look at logs and graphs, but look at business metrics and get details from customer support. Is anything off? If so, take a closer look.

Prioritize for follow-up work, ahead of new projects. If all goes well, there will be no further work on the migration. But what if problems surface, such as glitches for certain consumers or corrupted data?

I suggest you treat wrapping up the migration – and fixing any issues coming up – ahead of starting new work. If the issues are minor, you might be able to resolve them with the help of the support engineer or oncall engineer, if your team has such a rotation. If they are more complex issues, you might have to push back plans for future projects in favor of finishing the migration.

7. The migration long-tail

In organizations with lots of services, it's often impractical to migrate all consumers to a new system. Instead, a writeback strategy is used and the migration is broken up into these steps:

1. **Migrate so the new system is primary.** This new system uses writebacks or a similar approach to keep the old system up-to-date and usable by existing clients.
2. **The migration long-tail.** Migrate existing consumers, one by one, to the new system. Do this either via the teams migrating as self-service, the migrating team providing hand-holding, or as a mix.
3. **Shut down the old system.** Once all consumers are moved over, retire the old system. This step is often much later than most teams predict.

I've consistently observed the long-tail of migrations to take more time and more toll on teams owning the migration, than they expected. This often hits team morale as migrations are thankless enough to do, but they're especially taxing when they drag on, and the team gets questioned on why it's taking so long.

The risk of outages during this long-tail migration is high. It's common to proceed with great caution during the first phase of migrating to the primary system. However, during the long tail, teams usually proceed with fewer guard rails, and outages frequently occur as consumers migrate.

Also, the longer the old system is kept alive, the more problems occur because of the difference in functionality in the old versus the new system. This is especially problematic if code changes make it into the new system to support teams, which have yet to migrate to the new system.

How do you manage this migration long-tail, to make it as efficient as possible? Here are things to do:

1. **Put monitoring in-place for the writebacks to the old system.** If the primary system has issues writing back to the old system, this can lead to major outages, as it

did in the case of Uber. Even though the writeback is temporary, it's critical you monitor its health.

2. Make it easy for teams to migrate using self-service tooling. Create runbooks, tools to do dry-run migrations and other ways for engineers owning systems to migrate, test and execute the migrations themselves. Adopt the [platform team mindset](#) by enabling the consumers to execute the migration. And if your team owns a major migration, chances are you're either a platform team, or playing at being a platform team role during this migration.

3. Create visibility on the long-tail migration. Have visibility on:

- **Who** uses the old service? Which consumers are dependent on it?
- **Which** features do they use in the old service? This information will help with prioritizing migration, especially if the new service is missing functionality from the old service.
- **How much** do they use it? How often, or with what load?
- **How critical** is the old service for each team? If the old system breaks, which issues would it cause?
- **When** does each consumer plan to migrate? Is this migration in their roadmap? Is this a soft commitment or a hard one?

4. Get leadership support for the long-tail of the migration. I remember how challenging it was within Uber to motivate teams to move their systems over to the new payments system, instead of relying on the old systems that worked fine, thanks to writebacks. Most teams prioritized shipping business functionality over doing a migration that made no difference to their business.

It's true that migrations rarely count in business impact terms for customers which are forced to do them. And it's fair that they won't prioritize them when they don't think the work is important enough. This is where leadership comes in.

If a migration is truly strategic to a company, make leadership – up to and including the CTO or the engineering lead of the organization – understand why it's strategic. And

get them to prioritize the migration as one of the top initiatives for the organization during [annual or bi-annual planning](#).

This was exactly how much of the migration effort to move to Uber's new payments system was prioritized, this move became one of the top strategic priorities for the migration. Suddenly teams had a valid reason to prioritize this work, ahead of some business initiatives.

Without leadership support, large migrations will drag out, and some will never get done. So get this support as early as you can.

8. A migration checklist

I created an extensive migration preparation checklist. Feel free to use this document as inspiration, tailoring it to the needs of your team and organization. [Access the document here.](#)

4. Migration checklist

Step	Who	Status
Preparation		
Write the migrations plan	TBD	
Share the migrations plan with impacted stakeholders	TBD	
Sign off the migrations plan	(Define which stakeholders need to confirm)	
Pre-migration <i>Remove or add steps as needed</i>		
Monitoring: build it	TBD	
Validate monitoring working as expected	TBD	
Shadowing completion	TBD	
Load test completion	TBD	
Performance sampling completion	TBD	
Notify stakeholders on migration timeline		
Migration		
Perform the migration	TBD	
After the migration		

An excerpt from the migration checklist.

Part 3: The people and the business side

9. The people aspect of migrations

On top of the technical challenges, the people aspect is often just as difficult when executing migrations. Here are the most common challenges.

Who is leading the migration? If there is no clear answer to this question, the migration will be painful. The answer to this question should ideally be an individual, and a team around the individual.

I prefer to treat all migrations like an engineering project, one with a project lead and project members. Well-executed migrations should follow a similar runbook to [how software engineers lead projects](#), including planning, a kickoff, and closing off the migration.

Technical Program Managers (TPMs) leading migrations is often an excellent choice in organizations which have this role in place. As I wrote in the article [What TPMs do and what software engineers can learn from them](#):

“TPMs owning migrations and long-term tech debt efforts is common. If a migration or paying down tech debt can be done within a team, that team owns this effort. But what about the more common case where a migration impacts dozens of teams, or paying off tech debt needs several teams to move in parallel? Without dedicated ownership, these efforts fall between the cracks.

“TPMs tend to take ownership of these engineering efforts that need an owner, but for which an engineering team might not be the best owner because their incentives might be to move on quickly, rather than do what needs to be done.”

Who is the sponsor of the migration? This question becomes critical for migrations at large companies, where teams in different organizations need to do migration work. The sponsor of the migration would ideally be an engineering manager, director, or executive who heads up all of these teams. The sponsor prioritizes the migration, and nudges teams to get the migration work done.

If the sponsor is outside the organization, things get more challenging. Say the sponsor of the migration is the Engineering Director for Payments, but teams in the Customer Support, and Marketplace organizations need to do significant migration work. This sponsor then needs to influence the directors heading up the Customer Support and Marketplace teams, and get those directors to prioritize the migration ahead of some of their other projects.

We have so far avoided the term “politics” in this guide, but when it comes to large engineering organizations, we can’t tip-toe around the fact that politics matters in making large migrations successful. Well-connected engineering leaders who are good

at influencing their peers will more likely make migrations on their teams a success, than those leaders who don't have these skills or connections.

Attrition because of migrations is another common outcome when migrations drag on. Software engineers usually join teams to do interesting and exciting work. A migration taking twelve months is anything but, meaning some of these engineers will transfer to other teams, or even leave the company for different, more exciting work.

What can you do as an engineering manager to reduce attrition risk? Quite a few things, actually:

- **Don't stretch out the migration.** Use your political capital to get other teams to move quickly on the migration.
- **Create and celebrate ongoing migration wins.** Instead of having a never-ending project that feels disappointing for engineers working on it, create several, smaller projects which are all part of the migration, and each milestone is cause to celebrate.
- **Rotate people on and off the migration project.** Working on the same migration for a long period of time can lead to "migration burnout". Rotate your team so people work on other, non-migration projects. This is especially important if your migration long-tail is measured in many months, or even years.
- **Advocate for migrations to be recognized in performance reviews and promotions.** If those working on migrations never get promoted for this work, you'll soon find engineers wanting to avoid migration work in order to get ahead in their career, or even leaving your team if migration is all the work your team does. This would be a bad outcome for the company, given that well-executed migrations are important for any fast-growing company. So advocate for migration work to be recognized on a par with feature development work, both in promotions and performance reviews.

10. Selling migrations to leadership

It's common for leadership to not understand the value of migrations, and not being supportive in prioritizing these. What are ways to make the value of migrations more

clear and get more support from above?

Make the impact of the migration concrete. The best way is to start with the facts: what happens if this migration is done? What happens if it's deprioritized?

- Articulate the impact of the migration. Use numbers as much as possible. What will get better and by how much?
- Articulate the impact of NOT doing the migration at all. What will happen if the migration is never done? Try to use numbers in this estimation, as well.
- The impact of delaying the migration. What happens if the migration work starts 6 months later? What if it's only next year? In two years' time? Spell out how important or unimportant it is to start the migration as soon as possible.

Include engineering morale and attrition as impact. One aspect many engineering leaders leave out of migration estimates is the effect on their teams if the migration doesn't happen.

At companies that de-prioritize migrating systems and keep patching them, the engineering culture deteriorates. It's hard and frustrating for engineers to get things done. More engineers leave and it becomes harder to hire, and even those who are hired often leave early. Skipping migrations deteriorates all of engineering, over time.

It's fair to project an increase in attrition if the migration does not happen, and this percentage could climb, as time passes. For your own team, you can bring more specific numbers; you might include key people who may leave if leadership offers no support for projects that improve the reliability, maintainability or engineering culture of the company.

Influence your peers to support the migration. You might find yourself in situations where you need additional support to put the migration on the organization's roadmap. Instead of going straight to the organization leader, start with getting your peers on the same page. When most of them support doing the migration and getting it done soon, then use this support to add the migration project to the roadmap.

Ask for forgiveness, nor permission. So far, most of my advice has focused on getting support from your peers and your leadership, for the go-ahead to do a migration. But what if this doesn't work?

Sometimes you should do the right thing for the organization and company, and ask for forgiveness later.

If you're an engineering manager, you might decide to kick off the migration, and invest more of your team's time in making it a success. Or you could do a part of the migration and gather evidence on its importance.

If the migration is a major win, you might be fine going ahead with it. If you work at a company which values leaders taking initiative, then be the leader who takes the important initiative!

11. Closing advice for migrations

Show me how you do migrations, and I'll tell you how solid your engineering culture is. All fast-growing companies need to become excellent at starting and completing migrations.

As Netflix software engineer Lorin Hochstein [phrased it](#):

"Platform engineering" orgs should really call themselves "migration engineering." Because doing migrations needs to be the org's core competency."

I cannot think of any successful high-growth company which has not become great at doing migrations at scale, it's a competency that often goes hand-in-hand with [platform teams](#) and [TPMs](#).

In this issue we covered a blueprint you can use as a starter for the migrations you undertake. I encourage you to treat this article as a reference, and come back to it when working on migrations.

If I had to highlight a few key takeaways from this article, they are:

- **Treat migrations the same as you do important projects.** Get a team together, do the planning, the kickoff, and keep communicating about progress with interested teams.
- **Build tooling, even if it's throwaway tooling.** Most migrations go wrong in ways the engineering team doesn't notice, but customers do. Build monitoring and alerting so your team won't be that clueless team.
- **Do a zero-downtime migration if you've never done one before.** Zero-downtime migrations are expensive to do on one end, but few teams and people know how to do them well on the other. You won't know if a zero downtime migration is overkill or a convenient approach, if you don't know its tradeoffs. It's hard to know the tradeoffs if you've never tried doing one.
- **Prepare for a long-tail and don't let it drag on too long.** With many migrations, there will be a long-tail in the migration effort. Take the advice in this guide to both prepare for it and to keep it as short as possible.

With this, you're all set to start your next migration. May it be a success, one that consumers never notice and without a long-tail!

12. Further reading

Interested in learning more about migrations? Here are few of my recommendations:

- [Migrations: the sole scalable fix to tech debt](#) by Will Larson, CTO at Calm
- [Managing migrations](#) by Uma Chingunde, VP of Engineering at Render
- [Migrating functionality between large-scale prod systems](#) from Uber
- [The storefront rewrite](#) by Shopify. Interesting detail on how bugs needed to be re-implemented for compatibility.
- [Designing even larger applications](#) by Malte Ubl, formerly principal engineer at Google
- [Small safe steps migrations](#): a self-paced workshop by software engineer by Edu Ferro
- [A product-driven approach to automating migrations](#) from Two Sigma

- [Large-scale cloud migrations](#): a high-level overview by Google cloud.

Thanks to [Frank de Jonge](#) for his feedback and review on this article and to [Chris Peterson](#), [Simon Willison](#), [Lorin Hochstein](#) and several other people who have given feedback [on this Twitter thread](#).

How did you like this week's newsletter? 🤔

[Amazing](#) • [Great](#) • [Good](#) • [OK](#) • [So-so](#)



73 Likes

Comments



Write a comment...

© 2023 Gergely Orosz • [Privacy](#) • [Terms](#) • [Collection notice](#)
[Substack](#) is the home for great writing